Movie Collection API

Basil Wazwaz

ASE 230 Project 1

PHP, MySQL, and Bearer tokens

Project idea

This API stores a small list of movies and genres.

- A movie has a title, release year, and genre ID.
- A genre has a unique name.
- Both resources support create, read, update, and delete.
- Delete requests need a Bearer token.

The goal is to keep the project small and easy to test.

Database tables

Table	Fields
genres	id, name
movies	id, title, release_year, genre_id

`movies.genre_id` refers to `genres.id`.

A genre cannot be deleted while a movie still uses it.

The schema starts with two genres and two movies.

Database setup

Run from the folder containing README.md:

```
sudo mysql < code/schema.sql
sudo mysql
```

Create the `movie_app` user and grant SELECT, INSERT, UPDATE, and DELETE on `movie_collection.*`.

The full SQL and password setup are in README.md.
Import the schema once into a fresh database.

Start the PHP server

Set `DB_PASSWORD` to the database user's password.

Set a long random `API_TOKEN` in the same terminal.

```
export DB_HOST=127.0.0.1
export DB_USER=movie_app
# Set DB_PASSWORD and API_TOKEN as shown in README.md.
cd code
php -S localhost:8080 router.php
```

Base URL: `http://localhost:8080`

Press Ctrl+C in this terminal to stop the server.

Request format

POST and PUT requests use JSON.

```
curl -i -X POST http://localhost:8080/genres \
  -H 'Content-Type: application/json' \
  -d '{"name":"Comedy"}'
```

A successful create returns 201 and the new record.

Use the returned ID in later requests.

The next slides show example responses after a fresh import.

GET /genres

Lists all genres in ID order. No token is needed.

```
curl http://localhost:8080/genres
```

200 OK:

```
[{"id":1,"name":"Science Fiction"},
 {"id":2,"name":"Animation"}]
```

GET /genres/{id}

Gets one genre using its ID.

```
curl http://localhost:8080/genres/1
```

200 OK:

```
{"id":1,"name":"Science Fiction"}
```

An ID that does not exist returns 404.

POST /genres

Adds a genre. The name must be unique and nonempty.

```
curl -X POST http://localhost:8080/genres \
  -H 'Content-Type: application/json' \
  -d '{"name":"Comedy"}'
```

201 Created, example new ID:

```
{"id":3,"name":"Comedy"}
```

The name can be up to 80 bytes long.

PUT /genres/{id}

Changes an existing genre's name.

```
curl -X PUT http://localhost:8080/genres/3 \
  -H 'Content-Type: application/json' \
  -d '{"name":"Drama"}'
```

200 OK:

```
{"id":3,"name":"Drama"}
```

Use an ID returned by your own POST request.

DELETE /genres/{id}

Requires the configured Bearer token.

Delete the genre's movies first.

```
curl -X DELETE http://localhost:8080/genres/3 \
  -H "Authorization: Bearer $API_TOKEN"
```

200 OK:

```
{"message":"Record deleted."}
```

A genre still used by a movie returns 409.

GET /movies

Lists all movies in ID order. No token is needed.

```
curl http://localhost:8080/movies
```

200 OK:

```
[{"id":1,"title":"Interstellar",
  "release_year":2014,"genre_id":1},
 {"id":2,"title":"Spirited Away",
  "release_year":2001,"genre_id":2}]
```

GET /movies/{id}

Gets one movie using its ID.

```
curl http://localhost:8080/movies/1
```

200 OK:

```
{"id":1,"title":"Interstellar",
 "release_year":2014,"genre_id":1}
```

A missing movie returns a JSON error and status 404.

POST /movies

Adds a movie linked to an existing genre.

```
curl -X POST http://localhost:8080/movies \
  -H 'Content-Type: application/json' \
  -d '{"title":"New Movie","release_year":2020,"genre_id":1}'
```

201 Created, example new ID:

```
{"id":3,"title":"New Movie","release_year":2020,"genre_id":1}
```

Year and genre ID must be JSON integers, not quoted strings.

PUT /movies/{id}

Replaces the movie's editable fields. Send all three fields.

```
curl -X PUT http://localhost:8080/movies/3 \
  -H 'Content-Type: application/json' \
  -d '{"title":"Updated Movie","release_year":2021,"genre_id":2}'
```

200 OK:

```
{"id":3,"title":"Updated Movie","release_year":2021,"genre_id":2}
```

A later GET returns these saved values.

DELETE /movies/{id}

Requires the same token as the genre delete route.

```
curl -X DELETE http://localhost:8080/movies/3 \
  -H "Authorization: Bearer $API_TOKEN"
```

200 OK:

```
{"message":"Record deleted."}
```

A GET for this ID now returns 404.

Bearer token check

`auth.php` reads the Authorization header.

- The expected value comes from the server's `API_TOKEN`.
- `hash_equals` compares it with the supplied token.
- A missing or wrong token returns 401 before database access.
- No configured server token returns 503.

Example rejected response:

```
{"error":"A valid Bearer token is required."}
```

Source: PHP Manual, `hash_equals` (link in README.md).

Validation and errors

Status	Meaning
400 / 415	Invalid JSON / wrong content type
401	Missing or wrong delete token
404 / 405	Missing record or route / unsupported method
409	Duplicate genre or genre still in use
422	Invalid fields or unknown genre ID
500 / 503	Database failure / token not configured

Movie years must be 1888 to 2100. Titles cannot be blank.

PHP and database code

`index.php` reads the path and request method.
It validates fields, calls PDO, and returns JSON.

`db.php` opens the MySQL connection using local settings.
Values go through prepared statements.
Table names come only from the two allowed route names.

`auth.php` handles the two protected delete routes.
The project uses simple functions and no framework.

Source: PHP Manual, `PDO::prepare` (link in README.md).

cURL tests

Set the same token used by the server, then run:

```
bash code/test_curl.sh
```

The test file covers all 10 endpoints and error cases.
It checks saved updates, rejected deletes, and missing records.

Expected final line:

```
29 checks passed. Temporary test records deleted.
```

Browser tests

Open through the server:

`http://localhost:8080/test_client.html`

- Each endpoint has a request button and a JSON result.
- Enter the server token before clicking Run all tests.
- The suite checks all 10 endpoints and token failures.
- Successful tests remove their temporary movie and genre.

Expected result: **23 checks passed**.

Do not open this page as a local file.

Deployment and project limits

NGINX sends API requests to PHP-FPM.
The deployment tutorial shows the configuration and test captures.

The local NGINX URL is `http://localhost:8081`.
Only the test HTML page is directly served as a static file.

This is a classroom API with one shared delete token.
It does not have user accounts, a login screen, or public hosting.
Project 2 can rebuild the same routes in Laravel.

Sources

- Project 1 overview and rubric supplied with the assignment.
- PHP Manual: `PDO::prepare` and `hash_equals`.
- MySQL 8.0 Reference Manual: FOREIGN KEY Constraints.
- NGINX documentation: `ngx_http_fastcgi_module`.
- Marp: Markdown Presentation Ecosystem.

Full URLs are in README.md.

The submission example was used to check folder organization.

Its student/course API and example scores were not used.